

§ 6.3 邻接矩阵

6.3.1 原理

邻接矩阵 (adjacency matrix) 是图ADT最基本的实现方式, 使用方阵 $A[n][n]$ 表示由 n 个顶点构成的图, 其中每个单元, 各自负责描述一对顶点之间可能存在的邻接关系, 故此得名。

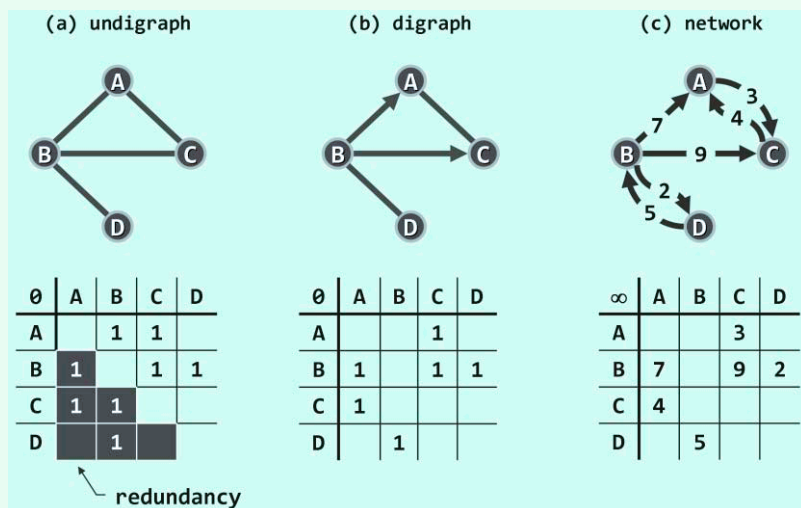


图6.5 邻接矩阵 (空白单元对应的边不存在, 其统一取值标注于矩阵最左上角)

对于无权图, 存在 (不存在) 从顶点 u 到 v 的边, 当且仅当 $A[u][v] = 1$ (0)。图6.5(a)和(b)即为无向图和有向图的邻接矩阵实例。

这一表示方式, 不难推广至带权网络。此时如图(c)所示, 矩阵各单元可从布尔型改为整型或浮点型, 记录所对应边的权重。对于不存在的边, 通常统一取值为 ∞ 或 0 。

6.3.2 实现

基于以上原理与构思实现的图结构如代码6.2所示。

```
1 #include "../Vector/Vector.h" //引入向量
2 #include "../Graph/Graph.h" //引入图ADT
3
4 template <typename Tv> struct Vertex { //顶点对象 (为简化起见, 并未严格封装)
5     Tv data; int inDegree, outDegree; VStatus status; //数据、出入度数、状态
6     int dTime, fTime; //时间标签
7     int parent; int priority; //在遍历树中的父节点、优先级数
8     Vertex(Tv const& d = (Tv) 0) : //构造新顶点
9         data(d), inDegree(0), outDegree(0), status(UNDISCOVERED),
10         dTime(-1), fTime(-1), parent(-1), priority(INT_MAX) {} //暂不考虑权重溢出
11 };
12
13 template <typename Te> struct Edge { //边对象 (为简化起见, 并未严格封装)
14     Te data; int weight; EStatus status; //数据、权重、状态
15     Edge(Te const& d, int w) : data(d), weight(w), status(UNDETERMINED) {} //构造新边
16 };
17
```

```

18 template <typename Tv, typename Te> //顶点类型、边类型
19 class GraphMatrix : public Graph<Tv, Te> { //基于向量，以邻接矩阵形式实现的图
20 private:
21     Vector<Vertex<Tv>> V; //顶点集（向量）
22     Vector<Vector<Edge<Te>*>> E; //边集（邻接矩阵）
23 public:
24     GraphMatrix() { n = e = 0; } //构造
25     ~GraphMatrix() { //析构
26         for (int j = 0; j < n; j++) //所有动态创建的
27             for (int k = 0; k < n; k++) //边记录
28                 delete E[j][k]; //逐条清除
29     }
30 // 顶点的基本操作：查询第i个顶点 (0 ≤ i < n)
31     virtual Tv& vertex(int i) { return V[i].data; } //数据
32     virtual int inDegree(int i) { return V[i].inDegree; } //入度
33     virtual int outDegree(int i) { return V[i].outDegree; } //出度
34     virtual int firstNbr(int i) { return nextNbr(i, n); } //首个邻接顶点
35     virtual int nextNbr(int i, int j) //相对于顶点j的下一邻接顶点
36     { while ((-1 < j) && (!exists(i, --j))); return j; } //逆向线性试探（改用邻接表可提高效率）
37     virtual VStatus& status(int i) { return V[i].status; } //状态
38     virtual int& dTime(int i) { return V[i].dTime; } //时间标签dTime
39     virtual int& fTime(int i) { return V[i].fTime; } //时间标签fTime
40     virtual int& parent(int i) { return V[i].parent; } //在遍历树中的父亲
41     virtual int& priority(int i) { return V[i].priority; } //在遍历树中的优先级数
42 // 顶点的动态操作
43     virtual int insert(Tv const& vertex) { //插入顶点，返回编号
44         for (int j = 0; j < n; j++) E[j].insert(NULL); n++; //各顶点预留一条潜在的关联边
45         E.insert(Vector<Edge<Te>*>(n, n, (Edge<Te>*) NULL)); //创建新顶点对应的边向量
46         return V.insert(Vertex<Tv>(vertex)); //顶点向量增加一个顶点
47     }
48     virtual Tv remove(int i) { //删除第i个顶点及其关联边 (0 ≤ i < n)
49         for (int j = 0; j < n; j++) //所有出边
50             if (exists(i, j)) { delete E[i][j]; V[j].inDegree--; } //逐条删除
51         E.remove(i); n--; //删除第i行
52         for (int j = 0; j < n; j++) //所有出边
53             if (exists(j, i)) { delete E[j].remove(i); V[j].outDegree--; } //逐条删除
54         Tv vBak = vertex(i); V.remove(i); //删除顶点i
55         return vBak; //返回被删除顶点的信息
56     }
57 // 边的确认操作
58     virtual bool exists(int i, int j) //边(i, j)是否存在
59     { return (0 ≤ i) && (i < n) && (0 ≤ j) && (j < n) && E[i][j] != NULL; }
60 // 边的基本操作：查询顶点i与j之间的联边 (0 ≤ i, j < n且exists(i, j))

```

```

61  virtual EStatus& status(int i, int j) { return E[i][j]->status; } //边(i, j)的状态
62  virtual Te& edge(int i, int j) { return E[i][j]->data; } //边(i, j)的数据
63  virtual int& weight(int i, int j) { return E[i][j]->weight; } //边(i, j)的权重
64  // 边的动态操作
65  virtual void insert(Te const& edge, int w, int i, int j) { //插入权重为w的边e = (i, j)
66      if (exists(i, j)) return; //确保该边尚不存在
67      E[i][j] = new Edge<Te>(edge, w); //创建新边
68      e++; V[i].outDegree++; V[j].inDegree++; //更新边计数与关联顶点的度数
69  }
70  virtual Te remove(int i, int j) { //删除顶点i和j之间的联边(exists(i, j))
71      Te eBak = edge(i, j); delete E[i][j]; E[i][j] = NULL; //备份后删除边记录
72      e--; V[i].outDegree--; V[j].inDegree--; //更新边计数与关联顶点的度数
73      return eBak; //返回被删除边的信息
74  }
75 };

```

代码6.2 基于邻接矩阵实现的图结构

可见，这里利用第2章实现并封装的**Vector**结构，在内部将所有顶点组织为一个向量**V[]**；同时通过嵌套定义，将所有（潜在的）边组织为一个二维向量**E[][]**——亦即邻接矩阵。

每个顶点统一表示为**Vertex**对象，每条边统一表示为**Edge**对象。

边对象的属性**weight**统一简化为整型，既可用于表示无权图，亦可表示带权网络。

6.3.3 时间性能

按照代码6.2的实现方式，各顶点的编号可直接转换为其在邻接矩阵中对应的秩，从而使得图ADT中所有的静态操作接口，均只需 $O(1)$ 时间——这主要是得益于向量“循序访问”的特长与优势。另外，边的静态和动态操作也仅需 $O(1)$ 时间——其代价是邻接矩阵的空间冗余。

然而，这种方法并非完美无缺。其不足主要体现在，顶点的动态操作接口均十分耗时。为了插入新的顶点，顶点集向量**V[]**需要添加一个元素；边集向量**E[][]**也需要增加一行，且每行都需要添加一个元素。顶点删除操作，亦与此类似。不难看出，这些恰恰也是向量结构固有的不足。

好在通常的算法中，顶点的动态操作远少于其它操作。而且，即便计入向量扩容的代价，就分摊意义而言，单次操作的耗时亦不过 $O(n)$ （习题[6-2]）。

6.3.4 空间性能

上述实现方式所用空间，主要消耗于邻接矩阵，亦即其中的二维边集向量**E[][]**。每个**Edge**对象虽需记录多项信息，但总体不过常数。根据2.4.4节的分析结论，**Vector**结构的装填因子始终不低于50%，故空间总量渐进地不超过 $O(n \times n) = O(n^2)$ 。

当然，对于无向图而言，仍有改进的余地。如图6.5(a)所示，无向图的邻接矩阵必为对称阵，其中除自环以外的每条边，都被重复地存放了两次。也就是说，近一半的单元都是冗余的。为消除这一缺点，可采用压缩存储等技巧，进一步提高空间利用率（习题[6-4]）。

§ 6.4 邻接表

6.4.1 原理

即便就有向图而言, $\Theta(n^2)$ 的空间亦有改进的余地。实际上, 如此大的空间足以容纳所有潜在的边。然而实际应用所处理的图, 所含的边通常远远少于 $\Theta(n^2)$ 。比如在平面图之类的稀疏图 (sparse graph) 中, 边数渐进地不超过 $\Theta(n)$, 仅与顶点总数大致相当 (习题[6-3])。

由此可见, 邻接矩阵的空间效率之所以低, 是因为其中大量单元所对应的边, 通常并未在图中出现。因静态空间管理策略导致的此类问题, 并非首次出现, 比如此前的2.4节, 就曾指出这类缺陷并试图改进。既然如此, 为何不仿照3.1节的思路, 将这里的向量替换为列表呢?

是的, 按照这一思路, 的确可以导出图结构的另一种表示与实现形式。

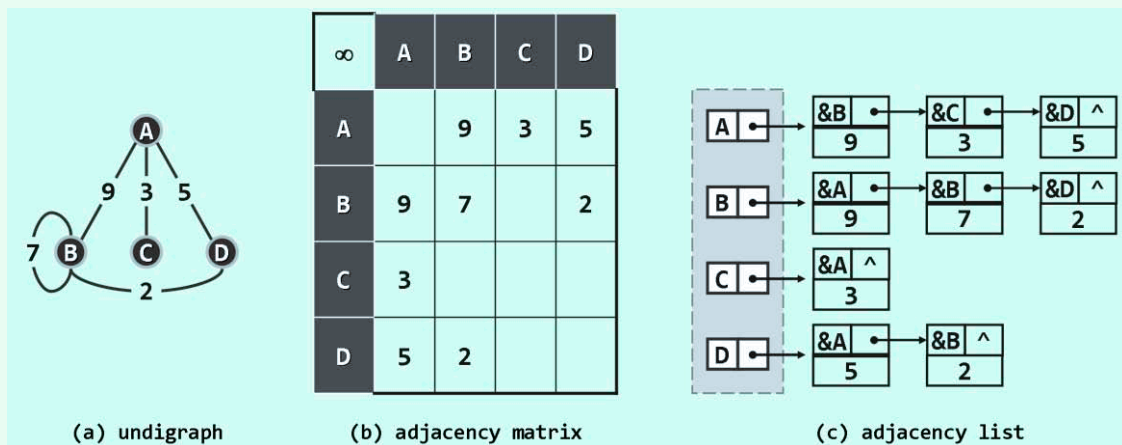


图6.6 以邻接表方式描述和实现图

以如图6.6(a)所示的无向图为例, 只需将如图(b)所示的邻接矩阵, 逐行地转换为如图(c)所示的一组列表, 即可分别记录各顶点的关联边 (或等价地, 邻接顶点)。这些列表, 也因此称作邻接表 (adjacency list)。实际上, 这种通用方法不难推广至有向图 (习题[6-5])。

6.4.2 复杂度

可见, 邻接表所含列表数等于顶点总数 n , 每条边在其中仅存放一次 (有向图) 或两次 (无向图), 故空间总量为 $\Theta(n + e)$, 与图自身的规模相当, 较之邻接矩阵有很大改进。

当然, 空间性能的这一改进, 需以某些方面时间性能的降低为代价。比如, 为判断顶点 v 到 u 的联边是否存在, $\text{exists}(v, u)$ 需在 v 对应的邻接表中顺序查找, 共需 $\Theta(n)$ 时间。

与顶点相关操作接口, 时间性能依然保持, 甚至有所提高。比如, 顶点的插入操作, 可在 $\Theta(1)$ 而不是 $\Theta(n)$ 时间内完成。当然, 顶点的删除操作, 仍需遍历所有邻接表, 共需 $\Theta(e)$ 时间。

尽管邻接表访问单条边的效率并不算高, 却十分擅长于以批量方式, 处理同一顶点的所有关联边。在以下图遍历等算法中, 这是典型的处理流程和模式。比如, 为枚举从顶点 v 发出的所有边, 现在仅需 $\Theta(1 + \text{outDegree}(v))$ 而非 $\Theta(n)$ 时间。故总体而言, 邻接表的效率较之邻接矩阵更高。因此, 本章对以下各算法的复杂度分析, 多以基于邻接表的实现方式为准。